DEBRIEFING DO PROJETO

ADEC
Painel de CI/CD

Como o trabalho funciona, quais tecnologias usa e qual a finalidade de cada uma, do commit até a produção.

Python 3.13 · FastAPI | React 19 · TypeScript · Vite | Docker · nginx | GitHub Container Registry | GitHub Actions | pytest · ESLint

DISCIPLINA
Abordagem DevOps e Entregas Contínuas

CURSO
ADS · FAETERJ

INTEGRANTES
alv4roso4res
GustaReis-bit
DrawnGZN

REPOSITÓRIO
alv4roso4res/adec

DATA
Novembro de 2026

1 O que é o trabalho

Trabalho final da disciplina **Abordagem DevOps e Entregas Contínuas**, feito em grupo de 3. A entrega é uma **apresentação de 15 minutos com demonstração ao vivo** a partir do repositório.

O objetivo é provar um **pipeline de CI/CD completo funcionando**:

- todo Pull Request é **testado automaticamente**;
- todo merge na `main` é testado e **entregue sozinho em produção**;
- a `main` é **protegida**: só aceita código por PR aprovado e com os testes passando.

Critérios de avaliação

CRITÉRIO	PESO
Pipeline de build e testes no PR	25%
Pipeline completo no merge, com entrega	25%
Branch protegida e merge bloqueado com teste falhando	15%
Explicação clara	20%
Participação de todos e tempo	15%

Regras importantes

- Repositório **público** no GitHub
- Build e **no mínimo 3 testes** automatizados
- Gatilho `pull_request`: build + testes
- Gatilho `push` na main: build + testes + **entrega**
- Cada integrante com **pelo menos 1 PR próprio**
- Só ferramentas gratuitas, sem cartão

2 O que a aplicação faz

Um painel de uma tela só que mostra o pipeline do próprio repositório, ao vivo. Ele faz parte da demonstração: enquanto o grupo abre um PR e faz o merge, o painel mostra tudo acontecendo.

CARD 1
Status da main
Último pipeline da main: estado, cronômetro, jobs e uma faixa por etapa.

CARD 3
PRs abertos
Autor, checks, aprovações e selo de estado de cada PR.

CARD 5
Versão publicada
Versão rodando × imagens no GHCR × último commit da main.

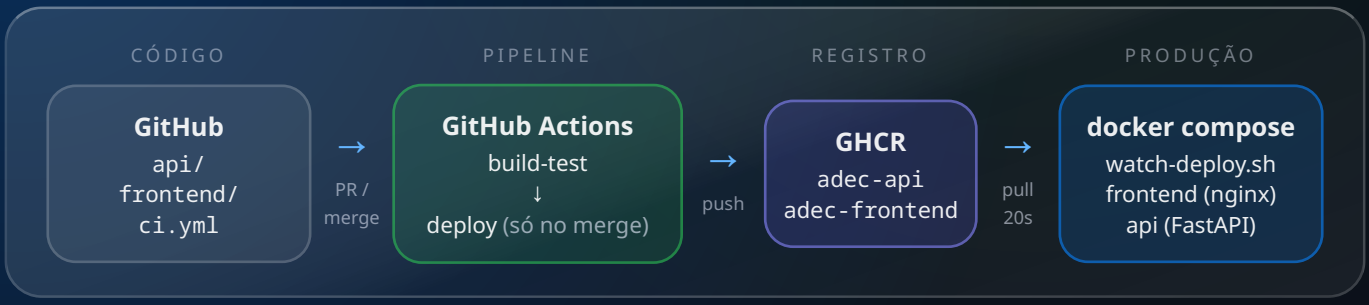
CARD 2
Últimos runs
Gráfico dos ~20 últimos runs, taxa de sucesso, duração média e deploys.

CARD 6
Commits da main
Últimos commits com o resultado do pipeline de cada um.

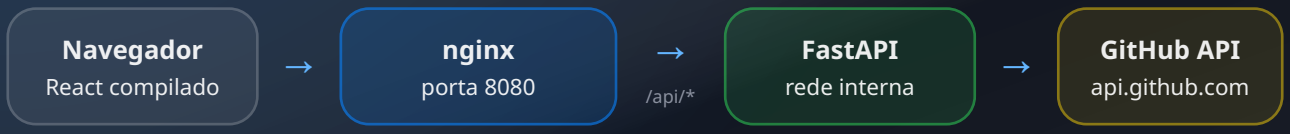
CARD 4
Integrantes
PRs e commits de cada um e se já cumpriu a regra do PR próprio.

3 Arquitetura

Três lugares: o código no GitHub, o pipeline no GitHub Actions e a produção na máquina do Alvaro.



O caminho de quem usa o painel



O frontend e a API ficam na mesma origem (o nginx repassa `/api/*`), então não há problema de CORS. A API nem expõe porta para fora: só o nginx fala com ela.

4 Tecnologias: backend

A API, na pasta `api/`.

TECNOLOGIA	PARA QUE SERVE NO PROJETO
Python 3.13	Linguagem do backend.
FastAPI	Framework web que cria as rotas (`/api/pipeline/current` , `/api/pulls` ...) e gera sozinho a documentação interativa em `/api/docs` .
Uvicorn	Servidor que executa o FastAPI e atende as requisições HTTP.
Pydantic	Define e valida o formato das respostas (schemas). Converte `head_sha` do Python em `headSha` no JSON.
pydantic-settings	Lê a configuração do `.env` (token, repositório, integrantes) com validação.
httpx	Cliente HTTP assíncrono que conversa com a API do GitHub.
Cache TTL + ETag	Guarda respostas por alguns segundos; depois pergunta ao GitHub "mudou?". Se não mudou (304), **não gasta o limite de requisições**.
uv	Gerenciador de pacotes do Python. O `uv.lock` trava as versões para o CI instalar exatamente o mesmo.
pytest · pytest-asyncio	Testes automatizados, inclusive de código assíncrono.
respx	Simula a API do GitHub nos testes: o CI testa sem internet e sem token.

Como o backend é organizado

rota recebe o pedido → **GitHubClient** busca + cache + erros → **service** função pura → **schema** formato da resposta

Os *services* nunca acessam a rede: recebem os dados crus do GitHub e devolvem o schema. Por isso os testes rodam com arquivos JSON de exemplo, sem rede e sem token.

5 Tecnologias: frontend

O painel, na pasta `frontend/`.

TECNOLOGIA	PARA QUE SERVE NO PROJETO
React 19	Biblioteca de interface: cada card é um componente reutilizável.
TypeScript	JavaScript com tipos. Os tipos espelham os schemas da API; se o formato do JSON mudar, o build acusa.
Vite	Servidor de desenvolvimento (`npm run dev`) e gerador do build. Também gera o `version.json`.
tsParticles	As partículas animadas do fundo.
CSS (Liquid Glass)	Visual de vidro estilo iOS: desfoque com `backdrop-filter`, brilho e aro de luz que seguem o mouse, inclinação 3D e efeito mola.
Hook `usePolling`	A atualização ao vivo: cada card busca a API sozinho; se falhar, mantém o último dado na tela.
ESLint	Verifica a qualidade do código; o CI reprova se houver erro.
npm	Gerenciador de pacotes do JavaScript; o `package-lock.json` trava as versões.

6 Tecnologias: infraestrutura e entrega

TECNOLOGIA	PARA QUE SERVE NO PROJETO
Docker	Empacota a API e o frontend em **imagens** que rodam igual em qualquer máquina. Os Dockerfiles são *multi-stage*: a imagem final leva só o necessário e roda sem usuário root.
Docker Compose	Sobe os dois containers juntos, com rede interna e *healthcheck* (o frontend espera a API ficar saudável).
nginx	Serve o React compilado e repassa `/api/*` para a API, na mesma origem.
GHCR	GitHub Container Registry: o "depósito" das imagens publicadas. É a **plataforma de entrega** do grupo.
GitHub Actions	O pipeline de CI/CD, definido em `.github/workflows/ci.yml`.
watch-deploy.sh	O "vigia" da produção: a cada 20s baixa a `latest` do GHCR e recria os containers que mudaram. É o deploy contínuo.
Branch protection	A `main` só aceita merge por PR, com **1 aprovação** e o check `build-test` verde, sem exceção nem para o administrador.
Git · GitHub CLI	Controle de versão e criação de PRs pelo terminal (`gh pr create`).

7 O pipeline por dentro

Arquivo `.github/workflows/ci.yml`.

Job build-test

Roda em todo PR e em todo merge.

1. Instala a API e roda o **pytest**.
2. Instala o frontend, roda o **lint** e o **build**.
3. Constrói as duas imagens Docker **sem publicar** (valida os Dockerfiles e aquece o cache).

Job deploy

Só no merge na main, e só se o build-test passou.

1. Faz login no GHCR.
2. Publica as duas imagens com a tag `sha-xxxxxxx`.
3. Só depois das **duas** publicadas, move a tag `latest`: a produção nunca pega API nova com frontend velho.

As duas tags das imagens

TAG	CARACTERÍSTICA	USO
sha-xxxxxxx	Imutável: identifica o commit exato	Rollback: `IMAGE_TAG=sha-xxxxxxx` no `.env`
latest	Móvel: sempre a última versão	É a que a produção acompanha

8 O ciclo completo de uma mudança

1. Um integrante cria uma **branch** e faz commits (ex.: `feat/frontend-painel`).
2. Faz **push** e abre um **Pull Request**.
3. O GitHub Actions roda o `build-test`; o painel mostra o PR com o selo “Aguardando checks”.
4. **Outro** integrante revisa e **aprova**. Ninguém aprova o próprio PR.
5. Com o check verde e a aprovação, é feito o **merge**.
6. O pipeline roda `build-test` e depois `deploy`, que publica as imagens no **GHCR**.
7. O `watch-deploy.sh` percebe a `latest` nova e **atualiza a produção sozinho**.
8. O painel mostra a versão nova no card 5.

Se algum teste falhar, o passo 5 fica **bloqueado**. Essa é a prova ao vivo da branch protegida.

9 Segurança e credenciais

CREDENCIAL	ONDE FICA	USO
Token classic `public_repo + read:packages`	`api/.env`, fora do git	A API lê os dados do GitHub e do GHCR
GITHUB_TOKEN do Actions	Gerado a cada run	O job `deploy` publica as imagens

Os arquivos `.env` estão no `.gitignore`: **nunca vão para o repositório**.

10 Roteiro da demonstração

15 minutos, todos os integrantes falam.

TEMPO	BLOCO	CONTEÚDO
2 min	**O que fizeram**	O painel e a combinação Python + React + GHCR.
3 min	**O que usaram**	As tecnologias e o porquê de cada uma.
5 min	**Como fizeram**	O `ci.yml` : gatilhos, os dois jobs, a condição do deploy e a branch protegida.
4 min	**Ao vivo**	(1) abrir um PR e mostrar o pipeline no Actions e no painel; (2) um PR com teste quebrado e o merge bloqueado; (3) o merge e o deploy chegando à produção.
1 min	**Dificuldades**	O que deu trabalho e como resolveram.

Atenção antes da apresentação: o integrante `DrawnGZN` ainda aparece “sem PR” no card de integrantes. Pela regra do trabalho, ele precisa abrir pelo menos um Pull Request próprio.

Resumo em uma frase

Cada mudança passa por **PR**, é **testada** pelo GitHub Actions, **aprovada** por outro integrante, **publicada** como imagem no GHCR e **entregue sozinha** em produção, e o próprio painel do grupo mostra tudo isso acontecendo ao vivo.